

Атрибути

`dir(foo) → foo.__dict__`

тоест ни показва речник само от собствените атрибути на обекта `foo`, ако има наследени атрибути и методи - те не са в речника

→ създава ни списък с имената на атрибутите и методите, които са налични за обекта `foo`. Включва и наследените от базови класове.

`getattr(foo, 'x') → foo.__getattr__('x') → foo.__getattribute__('x')`

търси и извлича стойността на 'x' на обекта `foo`, ако 'x' не съществува, -- `getattr` -- (ако е дефиниран), инак грешка

извиква се от всички опити да се достъпи атрибут, извиква се автоматично, когато достъпвам атрибут на обекта (независимо дали атр. съществува или не) трябва да има `super()`

извиква се само когато атрибута не съществува

`setattr(foo, 'x', 'y') → foo.__setattr__('x', 'y')`

използва се за задаване на новата стойност 'y' на атрибута 'x' на обекта `foo`. Извиква се -- `setattr` --, ако е дефиниран

извиква се винаги при задаване на стойност на атрибут, използва се `super()`, инак безкрайна рекурсия

`del foo.x → delattr(foo, 'x') → foo.__delattr__('x')`

искаме да изтрием атрибута `x` на обекта `foo`.

Извиква се -- `delattr` --, ако е дефиниран

еквивалентно на `del` извиква -- `delattr` --, ако го има

ако няма -- `delattr` -- и атрибута който искаме да изтрием → грешка

извиква се автоматично при използването на `del` и `delattr` част от метаобектния протокол има `super()` извиква се винаги при опит за изтриване

Мета - отнасящ се до себе си

Метаобекти - обекти, които описват обекти

Протокол - множество от правила в употреба при дадени обстоятелства

Метаобектен протокол - набор от специални методи, които управляват основното поведение на обектите

-- dunder -- - голяма част от метаобектния протокол
↓
всичко това са dunder-и

Кастове

Репрезентации

извежда документацията (doc string-ове) — `-- doc -- (self)` — потребителската репрезентация
`-- str -- (self)` — официалната репрезентация, връща низ
`-- dir -- (self)` — ако не е дефиниран, използва `-- герг --`, ако е написан
↓
списък с атрибути на обекта

Аритметика

Аритметичните операции се дефинират чрез дъндър методи. Това ни позволява да използваме дефинирането на поведението на операторите (+, -, *, /, //, %, **, @ (матрично умножение))

`-- add -- (self, other):`

...

Имаме и десни варианти на аритметичните операции.

Използват се, когато левият обект не поддържа тази операция (тоест дъндър метод), но десният обект може да извърши тази операция към левия обект.

`-- radd -- (self, other):`

...

Not Implemented → Когато операцията не е поддържана

ме и in-place варианти на аритметичните обекти:

Тоест не създават нов обект, а променят текущия. Например:

операторите $+=$, $-=$, $*=$...

--iadd-- (self, other): (собиране на място) $\equiv$ $+=$

...

$(x+y)$

винаги с приоритет лявата страна

първо търси --add-- на x

Ако го няма, търси --radd-- на y

Ако и двете ги няма $\rightarrow$ грешка

$(1+x)$

търси му --add--

Ако го няма търси --radd-- на x

$(1).add--(x)$

еквивалентно на $1+x$

$\rightarrow$ това е с приоритет

търси --add-- на 1 $\rightarrow$ връща NotImplemented

$(1).radd--(x)$

$\equiv x+1$

$\rightarrow$ търси --radd-- на 1

Унарна аритметика - извършват се само върху един

операнд

връщат нови обекти

- abs-- (self) $\rightarrow$ абсолютната стойност на обекта (модул)
- pos-- (self) $\rightarrow$ връща самия обект (унарен +) (не го променя)
- neg-- (self) $\rightarrow$ обръща знака на обекта (унарен минус)

Битова аритметика - има десни и in-place варианти

$a \& b \rightarrow$ I - проверява за --and-- на a

II - ако няма, проверява за --rand-- на b

III - ако няма, грешка

} връщат нов обект

$a \&= b \rightarrow$ търси за `--iand--` на a
променя обекта a

Равенство и хеширане

(is) `--eq-- (self, other)` - проверка на равенство между два обекта
(operator `==`)
(not is) `--ne-- (self, other)` - проверка на неравенство между два обекта
(operator `!=`)
`--hash-- (self)` $\rightarrow$ ^{за} генериране на хеш
стойност на обекта

! При дефиниране на `--eq--`, трябва да се дефинира и
`--hash--`, за да сме сигурни че поверението е правилно

Сравнения

- връщат bool

`--lt-- (self, other)` (`<`)
`--le-- (self, other)` (`<=`)
`--gt-- (self, other)` (`>`)
`--ge-- (self, other)` (`>=`)

$a < b \rightarrow a.\text{--lt--}(b)$

Нямаме десен вариант, защото `--lt--` $\rightarrow$ `--gt--`
`--le--` $\rightarrow$ `--ge--`

$a < b$ - I. търси `--lt--` на a

II. Ако го няма търси `--gt--` на b

Контекстен мениджър

with CM as име

`--enter-- (self)` $\rightarrow$ извиква се в началото

`--exit-- (self, type, value, traceback)` $\rightarrow$ извиква се
в края

Атрибути

`--dir-- (self)`

`--getattr-- (self, name)`

`--getatt-- (self, name)`

`--setattr-- (self, name, value)`

`--delattr-- (self, name)`

yield

- len -- (self) → len на обекта
- contains -- (self, item) → оператора in
- getitem -- (self, i) → достъп до елемент чрез индекс или ключ
- setitem -- (self, i, value) → задаване на стойности на елемент
- delitem -- (self, i) → изтриване на елемент

Итератори

- iter -- (self) → връща самия обект, който е итератор
- next -- (self) → връща следващият елемент от итерацията, когато няма повече елементи, хвърля StopIteration

Генераторите използват ключовата дума yield за автоматично създаване на итератор.

Метаатрибути

- dict -- → съдържа атрибутите на обекта като речник
- slots -- → за оптимизация на паметта в случаи с много обект.
↳ за ограничаване на атрибутите, които даден обект може да притежава
- class -- → сочи към класа, от който е създаден обекта
- globals -- → за функции, връща всички глобални променливи, видими за дадена функция
- name -- → връща името на обекта

Функции

- code -- → достъп до вътрешната структура на функцията и нейният байткод
- call -- (self, *args, **kwargs) → позволява извикването на обекти като функции

Импортиране на модули

`import module` $\equiv$ `module = --import--('module')`
използва се в $\swarrow$ $\searrow$ позволява динамично зареждане на модули и името на модула е известно по време на изпълнение
метапрограмирането

Конструиране

`--new--(cls, *args, **kwargs)` - статичен метод, който се извиква преди `--init--`
 $\downarrow$
истинския конструктор

създава и връща нов екземпляр на класа (или `super()....`)

`--init--(self, *args, **kwargs)` - инициализира обекта, създаден в `--new--`
 $\downarrow$
инициализатор

за задаване на начални стойности на атрибути на обекта
не връща нищо

Реконструиране

`--reduce--(self).`

определя как да бъде сериализиран и реконструиран обекта

`--reduce_ex--(self, protocol)`

подобно на `--reduce--`, но позволява съвместимост с различни версии на протокола за сериализация

Method resolution order (MRO)

`--mro--` - редът, по който търси методи или класове
- при множествено наследяване

Използва алгоритъм наречен C3 linearization

Други dunder's

`--sizeof--` - връща размера на обекта в байтове

`--base--` - връща базовия клас (родител) на даден клас

`Ellipsis` - тройна точка ...

криптори

Това са обекти, които предоставят персонализирана логика за управление на достъпа до атрибути.

`--get--(self, instance, owner)`

използва се за извличане на стойност на атрибут

`--set--(self, instance, value)`

използва се за задаване на стойност на атрибут

`--delete--(self, instance)`

използва се за изтриване на атрибут

`cached_property` - използваме го като декоратор

Импортираме го от `functools`

При първо извикване кодът се изпълнява и пресметнатата стойност се кешира в инстанцията. При следващото достъпване се взима от кеша.